Laporan Benchmark: ACS vs GenieACS

Beban simulator CPE identik pada kedua sistem · Kamis, 22 Oktober 2026

Sistem dibandingkan	**ACS kita** (Go + MariaDB + Redis, cabang `dev`, termasuk optimasi hari ini) vs **GenieACS 1.2.16** (Node.js + MongoDB 6, konfigurasi bawaan)
Beban	1.500 perangkat simulasi (profil ZTE TR-098), 64 sesi paralel, 1 registrasi + 4 putaran Inform periodik + 500 tugas `GetParameterValues`
Lingkungan	Satu laptop (8 CPU, 16 GB RAM, Docker 29.8.0 di Windows) yang menjalankan kedua ACS *dan* generator beban; kedua stack dibangun baru dengan database kosong, lalu diuji bergantian (ACS kita dulu, GenieACS kemudian)
Alat & cara ulang	`cmd/acsbench`, `scripts/bench.sh --devices 1500 --concurrency 64 --rounds 4 --tasks 500` (±10 menit); hasil mentah `bin/bench-*.json`

1. Kesimpulan

Dengan pekerjaan provisioning yang setara aktif di kedua sistem (diverifikasi di sisi CPE), ACS kita setara pada registrasi dan lebih cepat pada kondisi tunak dan fase tugas, dengan memori proses jauh lebih kecil. Belum terbukti pada perangkat nyata.

Dua run penuh berturut-turut (1.500 perangkat, 64 paralel): kondisi tunak 524–716 sesi/dtk vs 345–480 (p95 123–171 ms vs 211–302 ms); fase tugas 262–322 sesi/dtk vs 159–181 (p95 246–325 ms vs 589–680 ms). Registrasi: throughput GenieACS sedikit lebih tinggi (216–236 vs 197–223 sesi/dtk), p95 kita sedikit lebih baik — dianggap setara. Proses ACS memakai ±26–30 MiB (GenieACS ±597–632 MiB). Tidak ada galat pada kedua sistem.

322	**246 ms**	**26 MiB**	**0**
sesi/dtk fase tugas (run 1) (GenieACS 181)	p95 fase tugas (run 1) (GenieACS 589 ms)	memori proses ACS kita (GenieACS 632 MiB)	galat dari 8.000 sesi per sistem

Batas klaim (baca dulu). Beban berasal dari **simulator** CPE, bukan firmware nyata, dan semuanya berjalan di **satu laptop** yang juga menjalankan generator beban; stack yang tidak sedang diuji tetap hidup (idle), sehingga hasil dipengaruhi kondisi mesin. Hasil utama (bagian 3b) adalah **dua run penuh** dengan provisioning setara; tabel bagian 3 adalah **satu run tanpa provisioning**. Antar-run hari ini angka bergeser sampai ±30%, jadi selisih kecil tidak boleh dibaca sebagai pemenang — yang dianggap berarti hanya selisih yang searah pada kedua run. Konfigurasi bawaan dipakai di kedua sisi, dan **kompatibilitas ONT tidak dibuktikan oleh benchmark ini**.

2. Metode

- **Beban identik.** Alat yang sama (`cmd/acsbench`) mengirim urutan pesan CWMP yang sama ke kedua sistem. Tiap pekerja memakai satu koneksi TCP persisten, dan satu sesi CWMP selalu berada di satu koneksi seperti CPE nyata.
- **Gaya pesan.** SOAP ditulis dengan awalan namespace (`<soap-env:…>`) seperti CPE nyata. Tanpa itu GenieACS menolak Inform ("Unrecognized CWMP version"); ACS kita menerima kedua gaya.

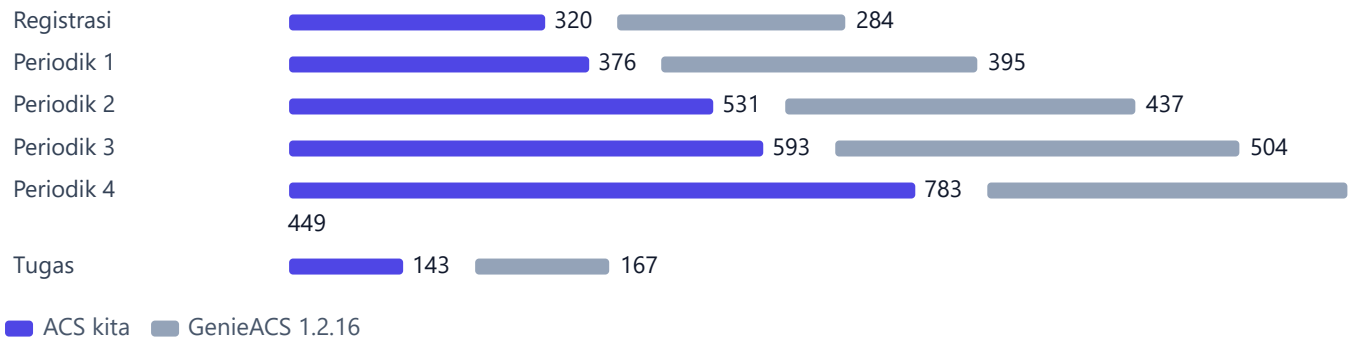
- **Fase.** (1) registrasi `0 BOOTSTRAP`; (2) empat putaran Inform periodik `2 PERIODIC`; (3) fase tugas: antre `GetParameterValues` lewat API, CPE menjawab dalam sesinya.
- **Latensi** = durasi satu sesi penuh (Inform → RPC → penutup) per perangkat. Persentil p50/p95/p99 dihitung dari seluruh sesi satu fase.
- **Parameter tugas** sengaja bukan yang ikut dilaporkan Inform: GenieACS menganggap nilai dari Inform masih segar dan melewati RPC, sedangkan ACS kita selalu bertanya ke CPE.
- **Sumber daya** disampel dari `docker stats` tiap 2 detik (hanya 5–6 sampel per run; 100% CPU = satu inti).

3. Hasil tanpa provisioning (satu run, sebelum optimasi lanjutan)

Sesi per detik: lebih tinggi lebih baik. Latensi: lebih rendah lebih baik. Sel hijau = lebih baik pada baris itu. 1.500 sesi per fase (500 pada fase tugas); 0 galat.

Fase	ACS kita				GenieACS 1.2.16			
	sesi/dtk	p50	p95	p99	sesi/dtk	p50	p95	p99
Registrasi	**320**	**187 ms**	**278 ms**	**309 ms**	284	191 ms	469 ms	628 ms
Periodik putaran 1	376	162 ms	**231 ms**	**279 ms**	**395**	**144 ms**	286 ms	399 ms
Periodik putaran 2	**531**	**101 ms**	**244 ms**	**287 ms**	437	127 ms	286 ms	371 ms
Periodik putaran 3	**593**	**102 ms**	**157 ms**	**181 ms**	504	111 ms	239 ms	329 ms
Periodik putaran 4	**783**	**78 ms**	**116 ms**	**137 ms**	449	124 ms	280 ms	361 ms
Tugas (antre API + sesi + RPC), 500	143	436 ms	**570 ms**	**627 ms**	**167**	**356 ms**	612 ms	690 ms

Throughput (sesi/detik)



Cara membaca

- **Registrasi dan putaran 1 (pemanasan)** setara; selisihnya masih dalam rentang kebisingan run.
- **Putaran 2–4 (kondisi tunak)**: ACS kita lebih tinggi 18–74% throughput-nya dan p95 lebih rendah 15–59%. Selisih ini searah pada ketiga putaran, tetapi GenieACS baru diukur **satu kali**; perbaikan jalur Inform di sisi kita sendiri terbukti konsisten pada run berulang (bagian 5).
- **Fase tugas (tanpa provisioning, run sebelum optimasi lanjutan)**: GenieACS unggul di throughput dan p50. Situasi ini berubah setelah optimasi lanjutan — lihat bagian 3b (hasil utama).

3b. Hasil utama: provisioning setara aktif di kedua sistem

Pekerjaan nyata yang sama diberikan ke kedua sistem: **paksa `PeriodicInformInterval=600` pada setiap Inform** + **tiga aturan/preset pengecoh** yang kondisinya dievaluasi tiap Inform tetapi tidak pernah cocok. ACS kita memakai **aturan otomasi** deklaratif; GenieACS memakai **provision JavaScript + preset**. Keberhasilan

diverifikasi di sisi CPE simulator: **1.500/1.500** perangkat menerima nilai pada kedua sistem (di run awal GenieACS sempat 1.499/1.500). Dua run penuh berturut-turut pada stack baru; sel = sesi/dtk · p95 (run 1 / run 2).

Fase	ACS kita	GenieACS 1.2.16	Selisih
Registrasi	223 · 406 ms / 197 · 409 ms	236 · 428 ms / 216 · 470 ms	setara (GenieACS +6–10% throughput; p95 kita lebih baik)
Periodik putaran 2	**680 · 140 ms / 669 · 143 ms**	424 · 237 ms / 345 · 302 ms	kita +58–94%
Periodik putaran 3	**716 · 124 ms / 589 · 152 ms**	417 · 275 ms / 383 · 266 ms	kita +54–72%
Periodik putaran 4	**664 · 159 ms / 524 · 171 ms**	480 · 211 ms / 404 · 249 ms	kita +30–38%
Tugas (500)	**322 · 246 ms / 262 · 325 ms**	181 · 589 ms / 159 · 680 ms	kita +65–78% throughput; p95 lebih rendah ±50%

- **Sumber daya (rata-rata CPU, run 1 / run 2):** ACS kita total ±457% / ±362% (acsd 179/141, MariaDB 262/206, Redis 16/15); GenieACS total ±545% / ±613% (proses CWMP 395/432, MongoDB 137/166, NBI 13/15). MariaDB tetap komponen terberat di sisi kita dibanding MongoDB.
- **Yang berubah sejak perbandingan setara pertama** (GenieACS saat itu masih unggul di tugas: 175 vs 141 sesi/dtk): (1) pencarian perangkat *persis* berdasarkan serial kini memakai indeks (`GET /devices?serial_number=`, migrasi 0041) — sebelumnya integrator hanya punya `search` substring yang memindai seluruh tabel, kelemahan produk nyata; (2) kuota perangkat tidak lagi menghitung `COUNT(*)` per registrasi untuk tenant tanpa batas (O(n) per registrasi = $O(n^2)$ pada onboarding massal); (3) INSERT perangkat lebih dulu, revive hanya bila duplikat; (4) tidak ada pencarian ganda untuk perangkat baru; (5) cache TTL pendek untuk kuota task dan tenant pemilik perangkat. **Jujur:** sebagian besar lonjakan fase tugas berasal dari (1) — klien benchmark ACS kita memang harus mencari id perangkat dari serial (GenieACS: id sudah mengandung serial), dan pencarian itu kini murah; panggilan tambahan itu tetap termasuk dalam waktu yang diukur.

4. Sumber daya

Komponen	CPU rata-rata	CPU puncak	Memori maks
ACS kita — proses `acsd`	159%	188%	**25 MiB**
ACS kita — MariaDB	259%	457%	165 MiB
ACS kita — Redis	15%	18%	6 MiB
GenieACS — `genieacs-cwmp`	334%	480%	589 MiB
GenieACS — MongoDB	174%	249%	416 MiB
GenieACS — NBI	9%	54%	190 MiB
Total ACS kita / GenieACS	≈433% / ≈517%	—	**≈196 MiB / ≈1.195 MiB**

- Proses ACS kita memakai **±24× lebih sedikit memori** dan ±2× lebih sedikit CPU daripada proses CWMP GenieACS untuk throughput yang sama atau lebih tinggi.
- **MariaDB adalah komponen terberat** di sisi kita (rata-rata ±2,6 inti) — lebih berat daripada MongoDB (±1,7 inti). Ini beban tulis per Inform (sesi, event, audit) dan masih menjadi kandidat optimasi berikutnya.
- Sampel hanya 5–6 titik per run, jadi persentase CPU adalah perkiraan kasar.

5. Optimasi yang dilakukan hari ini dan buktinya

Benchmark 21 Oktober menunjukkan ACS kita setara setelah pemanasan, tetapi lebih lambat saat registrasi dan fase tugas, dengan MariaDB berat. Penelusuran log query MariaDB menemukan tiap query dikirim sebagai tiga round trip (Prepare, Execute, Close) dan hot path Inform melakukan SELECT/UPDATE berulang.

Perubahan	Efek
Satu query = satu round trip (`interpolateParams` otomatis pada koneksi DB)	Hilang 2 round trip per query
Namespace CWMP ikut `INSERT` sesi baru	−1 UPDATE, −1 SELECT, −operasi Redis per sesi baru
`UpsertBatch` satu chunk tanpa `BEGIN/COMMIT`	−2 round trip dan −1 commit per Inform
Cache 5 detik untuk aturan ZTP dan preset enforce (dibuang langsung saat diubah di instance yang sama)	−2 SELECT per Inform
Event status task memakai task yang sudah dipegang	−3 SELECT per tugas

Query per sesi (MariaDB `Com_*`): SELECT 13,3 → 10,3 · UPDATE 3,5 → 2,5 · commit 1,5 → 0,2.

Pembandingan sebelum/sesudah pada kode kita sendiri (beban penuh, 2 run per sisi, urutan selang-seling)

Fase	Sebelum (sesi/dtk · p95), run 1 / run 2	Sesudah (sesi/dtk · p95), run 1 / run 2
Registrasi	144 · 704 ms / 160 · 750 ms	224 · 442 ms / 329 · 274 ms
Periodik putaran 4	310 · 342 ms / 403 · 247 ms	636 · 144 ms / 701 · 124 ms
Tugas	66 · 1.400 ms / 89 · 1.125 ms	79 · 1.483 ms / 108 · 813 ms

Jalur Inform ±1,7–2× lebih cepat dan p95 turun 2–3×; selisih antar-run jauh lebih kecil daripada selisih sebelum/sesudah. Fase tugas membaik tipis dan masih berisik. Verifikasi kode: `go vet` dan `go test ./...` hijau, suite e2e 30/30 lulus pada kode baru.

Kompromi: aturan ZTP/preset yang diubah lewat instance acsd lain baru terlihat setelah ≤5 detik. Lookup tenant per Inform sengaja tidak di-cache supaya penonaktifan tenant tetap langsung memutus akses.

6. Perbandingan fitur

Kemampuan GenieACS diverifikasi pada instans 1.2.16 yang berjalan (NBI menjawab 200 pada `/devices`, `/tasks`, `/presets`, `/provisions`, `/virtual_parameters`, `/files`, `/faults`, `/objects`).

Kemampuan	ACS kita	GenieACS 1.2.16
Multi-tenant, RBAC bertingkat, MFA TOTP, SSO OIDC	Ada	Peran dasar, tanpa tenant
Audit trail, notifikasi in-app/email, webhook, billing SaaS	Ada	Tidak ada
Tab WiFi, dashboard optik/suhu/uptime, retry task, "Refresh ONT", capture sesi dengan penyamaran rahasia	Ada	UI generik
Portal pelanggan akhir (self-service)	Ada	Tidak ada

Katalog vendor/path sebagai data; enkripsi rahasia task	Ada	Lewat skrip/provision
Virtual parameter (nilai turunan dari parameter perangkat)	Ada (22 Okt 2026) — ekspresi deklaratif, bukan skrip; per tenant + global	Ada
Provision berskrip (JavaScript)	Sebagian (22 Okt 2026) — **aturan otomasi kondisi–aksi** deklaratif, bukan skrip bebas: aksi tertutup, cooldown, suspend otomatis, dry-run, riwayat	Ada (JavaScript)
Query perangkat bebas + proyeksi + sort	Ada (22 Okt 2026) — kondisi nilai parameter, proyeksi, sort; tertutup & ter-bind	Ada
Resource `faults` tersendiri (daftar, hitung kejadian, tutup otomatis saat RPC berhasil)	Ada (22 Okt 2026)	Ada
Kematangan lintas firmware dan skala produksi	Baru 3 ONT fisik	Bertahun-tahun

7. Yang tidak diukur

- **Perangkat dan firmware nyata.** Uji ONT fisik 21 Oktober menemukan bug `xsi:type` yang tidak tertangkap simulator; benchmark ini tidak menggantikan uji lapangan.
- **Skala** ratusan ribu hingga jutaan perangkat, dan retensi data berhari-hari (pertumbuhan tabel, kinerja query lama).
- **Beban REST/UI bersamaan** dengan Inform, serta kegagalan jaringan dan restart komponen.
- **Pengaruh autentikasi.** ACS kita memverifikasi kredensial CWMP per tenant pada tiap Inform dan mencatat sesi, event, dan audit; GenieACS bawaan tanpa autentikasi CWMP. Sebagian selisih resource adalah harga fitur, bukan semata efisiensi.
- **GenieACS dengan tuning** (jumlah worker, indeks MongoDB, ekstensi) — yang diuji adalah konfigurasi bawaan.

8. Rekomendasi

1. **Jangan memakai benchmark ini sebagai klaim keandalan.** Klaim yang didukung: efisiensi memori dan throughput Inform setara-atau-lebih-baik pada beban simulator; fitur platform multi-tenant.
2. **Fase tugas sudah diprofil** dan dioptimasi (bagian 3b). Yang tersisa: registrasi massal (throughput masih setara, bukan unggul) dan tulis MariaDB per sesi.
3. **Kurangi beban tulis MariaDB per Inform** (event dan audit) bila skala perangkat mendekati ratusan ribu.
4. **Ulangi di mesin khusus** (server terpisah dari generator beban, beberapa run, median) bila angka akan dipakai di luar tim.
5. **Tutup gap fitur** yang relevan bagi pelanggan: JavaScript bebas sengaja tidak ada (risiko multi-tenant); bila kebutuhan nyata melampaui aturan otomasi deklaratif, langkah berikutnya JavaScript tersandbox — perlu keputusan dan uji keamanan tersendiri.

Sumber: `docs/pengujian/BENCHMARK_GENIEACS.md` (catatan teknis), `bin/bench-ours.json`, `bin/bench-genieacs.json`. PDF dibuat dari `BENCHMARK_GENIEACS.html` dengan `node scripts/build-docs-pdf.mjs BENCHMARK`. Sunting HTML-nya, bukan PDF.